VALIDATING AND PROCESSING USER INPUTS

Visual Basic 6.0 | Study notes for OAVS Computer Teacher competitive exam

Topics covered (syllabus order)

1. Overview
2. Importance
3. Types of validation
4. Implementing Form level and field level validations
5. Text box Properties
6. Using Events
7. Validate Event
+ Quick Revision

1. Overview

Definition. Validation is the process of checking the data entered by the user to confirm that it is complete, of the correct type, within the permitted range and in the correct format *before* the program uses it. **Processing** is the work the program does with the accepted data: converting, calculating, storing or displaying it.

Core concept. A program must never trust raw input. In VB 6.0 the commonest input control is the TextBox, and its `Text` property always returns what the user typed as a **String**. The sequence is therefore **accept input -> validate -> convert -> process -> output**.

Purpose. To let only correct, meaningful data reach the processing stage so that results are reliable and the program does not stop with a run-time error.

Working / process

- The user enters data in an input control (for example a TextBox).
- Validation code checks the data (blank? numeric? in range? correct format?).
- **Valid**: the data is converted to the required type and processed; the result is displayed.
- **Invalid**: an error message is shown, focus returns to the field and the user re-enters the data.

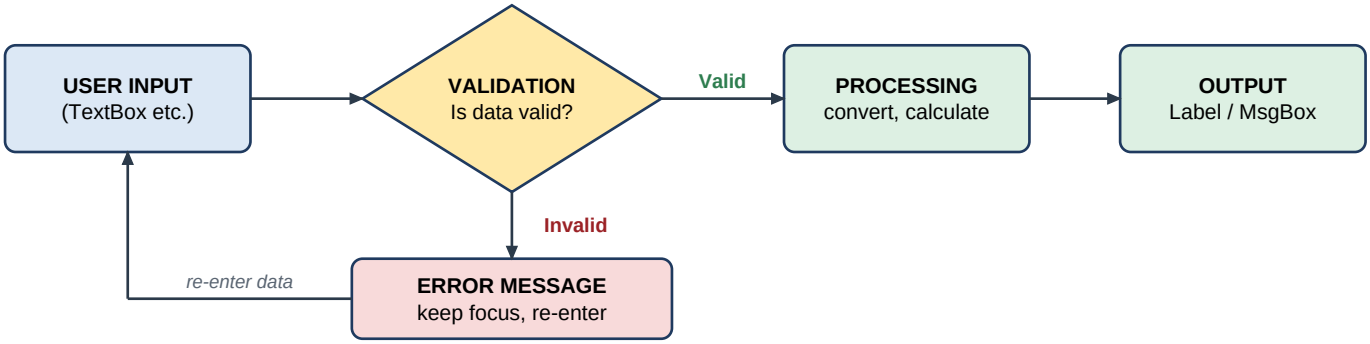

Figure 1: User input -> validation -> processing flow

VB 6.0 tools used (explained in later topics)

Kind	Tool	Used for
Property	`MaxLength`	Limits how many characters the user can type (length check)
Property	`CausesValidation`	Decides whether the Validate event fires when focus moves to a control
Property	`Locked`	Makes a TextBox read-only so the user cannot change it
Property	`PasswordChar`	Shows a mask character (e.g. *) instead of the typed characters
Event	`KeyPress`	Checks or rejects each key as it is typed
Event	`Change`	Runs when the text changes; used for live processing
Event	`LostFocus`	Checks a field after the user has left it
Event	`Validate`	Checks a field before focus leaves; `Cancel = True` keeps the user in it
Event	`Click`	Runs form-level validation and processing when a button is clicked
Function	`IsNumeric`	Tests whether a value can be treated as a number (data type check)
Function	`Len`	Returns the number of characters (length and blank check)
Function	`Trim`	Removes leading and trailing spaces so a blank entry is detected
Function	`Val`	Converts text to a number for calculation (it does not validate)

Program with output

PROGRAM 1: Area of a square with input validation

Controls on the form: txtSide (TextBox), cmdArea (CommandButton), lblArea (Label)

```
Private Sub cmdArea_Click()
    If IsNumeric(txtSide.Text) Then                  ' validation
        lblArea.Caption = Val(txtSide.Text) ^ 2      ' processing
    Else
        MsgBox "Please enter a number"
    End If
End Sub
```

OUTPUT

Input / action	Output displayed	Shown in
Side = 5, click Area	lblArea shows **25**	Label
Side = 2.5, click Area	lblArea shows **6.25**	Label
Side = five, click Area	Message: "Please enter a number"; lblArea is unchanged	Message box
Side left blank, click Area	Message: "Please enter a number" (IsNumeric("") is False)	Message box

OAVS EXAM POINTS

- Validation is always done **before** processing.
- Validation can be at keystroke, field or form level (Topic 4).
- The `Text` property of a TextBox always holds **String** data.
- Validation checks that data is **reasonable**; it cannot prove that data is **true** (age 25 is accepted even if the real age is 30).

COMMON MISTAKES / CONFUSING POINTS

- **Validation vs verification**: validation checks sensible format/range; verification checks against the source or by re-entry (e.g. retyping a password).
- Using TextBox text directly in calculations without converting/validating it first.

SUMMARY. Validation checks input before use; processing then converts and uses valid data. Flow: input, validate, process, output - with an error loop for invalid data.

2. Importance

Definition and core concept. Input validation is the **first line of defence** of an application. Users make typing mistakes, leave fields blank or enter unexpected values; validation makes sure such data never reaches the processing stage.

Purpose. Correct results, a stable program and a user-friendly interface.

Why input validation is required

- **Prevents run-time errors** caused by wrong data types, blanks, zero divisors or large values.
- **Maintains data integrity**: **GIGO** (Garbage In, Garbage Out) - wrong input gives wrong output.
- **Gives reliable results**: marks of 150 or age of -5 would produce meaningless output.
- **Improves user experience**: clear, early messages help the user correct mistakes immediately.
- **Protects later stages**: only clean data is stored, printed or passed to other procedures.
- **Limits unexpected input**, e.g. over-long text (`MaxLength`) or illegal characters.

What happens without validation

Bad input	Statement executed	Result
"abc"	`CInt(txtNum.Text)`	Run-time error **13** - Type mismatch
"" (blank)	`CInt(txtNum.Text)`	Run-time error **13** - Type mismatch
"40000"	`CInt(txtNum.Text)`	Run-time error **6** - Overflow (Integer maximum is 32767)
Divisor = 0	`x = a / b`	Run-time error **11** - Division by zero
Age = -5	No check	Accepted silently: a logical error (GIGO)

Programs with output: without and with validation

PROGRAM 2: Without validation (causes run-time errors)

Controls on the form: txtNum (TextBox), cmdOK (CommandButton)

```
Private Sub cmdOK_Click()
    Dim n As Integer
    n = CInt(txtNum.Text)
    MsgBox "Number = " & n
End Sub
```

OUTPUT

Input / action	Output displayed	Shown in
txtNum = 25	Message: "Number = 25"	Message box
txtNum = abc	Run-time error '13': Type mismatch; the program stops (End / Debug buttons)	Error dialog
txtNum left blank	Run-time error '13': Type mismatch	Error dialog
txtNum = 40000	Run-time error '6': Overflow	Error dialog

PROGRAM 3: With validation (program keeps running)

Controls on the form: txtNum (TextBox), cmdOK (CommandButton)

```
Private Sub cmdOK_Click()
    Dim n As Integer
    If IsNumeric(txtNum.Text) Then
        n = CInt(txtNum.Text)
        MsgBox "Number = " & n
    Else
        MsgBox "Please enter a number"
    End If
End Sub
```

OUTPUT

Input / action	Output displayed	Shown in
txtNum = 25	Message: "Number = 25"	Message box
txtNum = abc	Message: "Please enter a number"; program continues	Message box
txtNum left blank	Message: "Please enter a number"; program continues	Message box
txtNum = 40000	Still Overflow error 6: a type check alone is not enough; a **range check** is also needed (Topic 3)	Error dialog

OAVS EXAM POINTS

- **GIGO** = Garbage In, Garbage Out.
- Error numbers: **13** Type mismatch, **11** Division by zero, **6** Overflow.
- Validation is **preventive**: it stops bad data before it causes an error.

COMMON MISTAKES / CONFUSING POINTS

- Thinking validation is needed only for numeric data; text, dates and passwords need it too.
- A logical error (e.g. age -5) gives **no** run-time error, so only validation can catch it.

SUMMARY. Validation prevents run-time and logical errors, protects data integrity (GIGO) and makes the application robust and user-friendly.

3. Types of validation

Definition and core concept. A **type of validation** is a kind of check applied to the data. Real forms combine several checks, normally in this order: presence, data type, then range / length / format.

Purpose. Each type catches a different kind of mistake, so using the right type makes the check accurate and the message meaningful.

Type	What it checks	VB 6.0 technique	Example
Presence (required field)	Field is not left blank	`Len(Trim(txt.Text)) = 0`	Name = " " fails
Data type	Value is a number / date	`IsNumeric()`, `IsDate()`	IsNumeric("25") True; IsNumeric("2x") False
Range	Value lies between minimum and maximum	`Val(x) >= 0 And Val(x) <= 100`	Marks 105 fails (0 to 100)
Length	Number of characters	`Len()`, `MaxLength` property	PIN must be 4 characters
Format (pattern)	Characters follow a pattern	`Like` operator, `InStr()`	"560001" Like "######" True
Character (keystroke)	Only permitted keys are accepted while typing	`KeyPress` event, `KeyAscii`	Digits only in Age box
List (lookup)	Value belongs to an allowed set	`Select Case`, `InStr()`	Grade must be A, B or C
Consistency (cross-field)	Related fields agree	Compare the two fields	Password = Confirm password

`Like` wildcards: `?` any one character, `*` zero or more characters, `#` any one digit, `[abc]` any one character from the list.

Validation decision flowchart

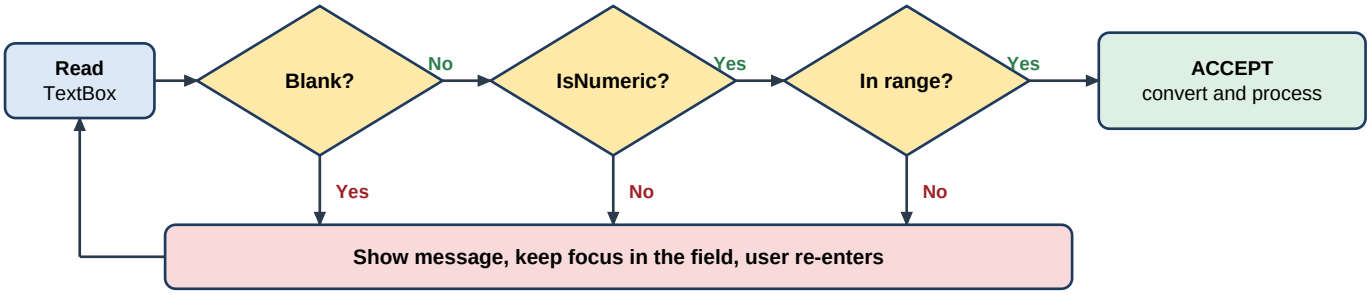

Figure 2: Validation decision flowchart (checks applied in order)

Programs with output

PROGRAM 4: Format check: 6-digit PIN code

Controls on the form: txtPin (TextBox), cmdCheckPin (CommandButton), lblMsg (Label)

```
Private Sub cmdCheckPin_Click()
    If txtPin.Text Like "######" Then
        lblMsg.Caption = "Valid PIN"
    Else
        lblMsg.Caption = "Invalid PIN"
    End If
End Sub
```

OUTPUT

Input / action	Output displayed	Shown in
txtPin = 560001	lblMsg shows **Valid PIN**	Label
txtPin = 56A001	lblMsg shows **Invalid PIN** (A is not a digit)	Label
txtPin = 5600	lblMsg shows **Invalid PIN** (only 4 characters)	Label
txtPin = 5600012	lblMsg shows **Invalid PIN** (7 characters)	Label

PROGRAM 5: Format check: simple e-mail address

Controls on the form: txtMail (TextBox), cmdCheckMail (CommandButton)

```
Private Sub cmdCheckMail_Click()
    If InStr(txtMail.Text, "@") = 0 Or InStr(txtMail.Text, ".") = 0 Then
        MsgBox "Invalid e-mail", vbExclamation
    Else
        MsgBox "E-mail accepted"
    End If
End Sub
```

OUTPUT

Input / action	Output displayed	Shown in
txtMail = ravi@mail.com	Message: "E-mail accepted"	Message box
txtMail = ravimail.com	Message: "Invalid e-mail" (no @)	Message box
txtMail = ravi@mailcom	Message: "Invalid e-mail" (no dot)	Message box
txtMail = a@.	Message: "E-mail accepted"; shows this is only a **basic** check	Message box

OAVS EXAM POINTS

- Do the **presence** check first, then data type, then range.
- A range check needs a numeric value first (type check before range check).
- `MaxLength` gives a length check **while typing**; it cannot enforce a minimum length.

COMMON MISTAKES / CONFUSING POINTS

- `Val` **is not validation**: `Val("abc")` = 0 and `Val("12abc")` = 12, with no error. It only converts.
- `IsNumeric` also returns True for "1e5", "$25", "1,200" and "&H1F"; it is not a strict digits-only test (use `KeyPress` or `Like`).
- Confusing a **length** check (how many characters) with a **range** check (how large a value).

SUMMARY. Main types: presence, data type, range, length, format, character, list and consistency checks. Apply them in a logical order and show a clear message for each failure.

4. Implementing Form level and field level validations

Definition and core concept. Field-level validation checks **one control** while the user types in it or when the user leaves it. **Form-level validation** checks **all controls together** when the user submits the form (OK / Save / Submit button).

Purpose. Field-level gives immediate feedback; form-level is the final safety net, so good applications use **both**.

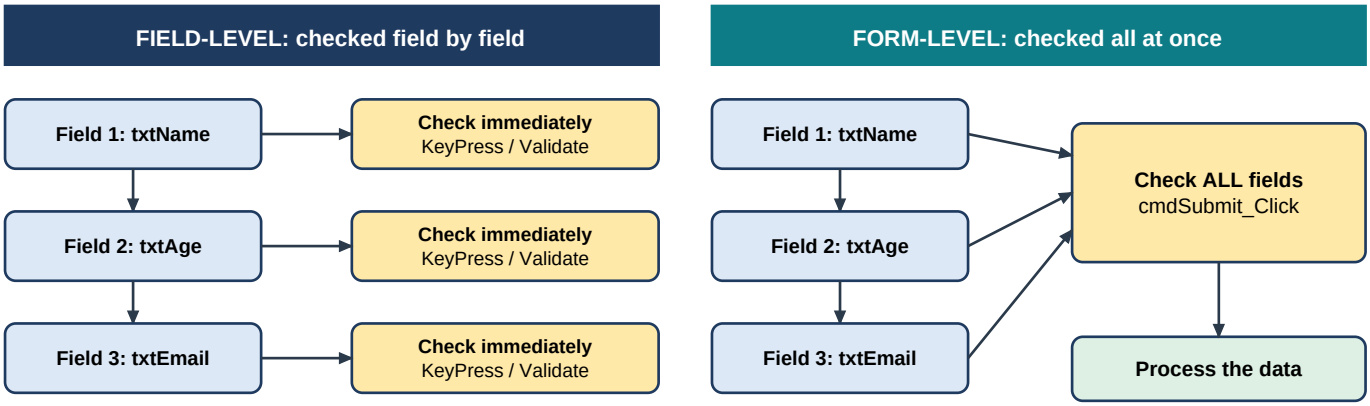

Figure 3: Form-level vs field-level validation

A. Field-level validation (events: KeyPress, Change, LostFocus, Validate)

PROGRAM 6: Field-level: digits only in Age (KeyPress event)

Controls on the form: txtAge (TextBox)

```
Private Sub txtAge_KeyPress(KeyAscii As Integer)
    If Not ((KeyAscii >= 48 And KeyAscii <= 57) Or KeyAscii = 8) Then
        KeyAscii = 0
    End If
End Sub
```

OUTPUT

Input / action	Output displayed	Shown in
Types A, 2, x, 5	txtAge shows **25** (A and x are ignored)	Text box
Then presses Backspace	txtAge shows **2** (Backspace, code 8, is allowed)	Text box
Types - or .	No change: both keys are ignored	Text box
Pastes "abc" from the right-click menu	txtAge shows **abc**: KeyPress does not catch pasted text	Text box

PROGRAM 7: Field-level: required Name (LostFocus event)

Controls on the form: txtName, txtAge (TextBoxes)

```
Private Sub txtName_LostFocus()
    If Len(Trim(txtName.Text)) = 0 Then
        MsgBox "Name is required", vbExclamation
        txtName.SetFocus
    End If
End Sub
```

OUTPUT

Input / action	Output displayed	Shown in
Name left blank, presses Tab	Message: "Name is required"; focus goes back to txtName	Message box
Name = three spaces, presses Tab	Message: "Name is required" (Trim removes the spaces)	Message box
Name = Asha, presses Tab	No message; focus moves to txtAge	No output

Note: using `SetFocus` inside `LostFocus` can cause focus problems; the `Validate` event (Topic 7) is the better field-level choice.

B. Form-level validation (event: Click of the command button)

PROGRAM 8: Form-level: validate Name and Age when Submit is clicked

Controls on the form: txtName, txtAge (TextBoxes), cmdSubmit (CommandButton), lblResult (Label)

```
Private Function IsFormValid() As Boolean
    IsFormValid = False
    If Len(Trim(txtName.Text)) = 0 Then
        MsgBox "Name is required", vbExclamation: txtName.SetFocus: Exit Function
    End If
    If Not IsNumeric(txtAge.Text) Then
        MsgBox "Age must be numeric", vbExclamation: txtAge.SetFocus: Exit Function
    End If
    If Val(txtAge.Text) < 1 Or Val(txtAge.Text) > 100 Then
        MsgBox "Age must be 1 to 100", vbExclamation: txtAge.SetFocus: Exit Function
    End If
    IsFormValid = True
End Function

Private Sub cmdSubmit_Click()
    If IsFormValid() Then
        lblResult.Caption = "Welcome " & Trim(txtName.Text) & ", age " & Val(txtAge.Text)
    End If
End Sub
```

OUTPUT

Input / action	Output displayed	Shown in
Name blank, any Age, click Submit	Message: "Name is required"; focus to txtName (only the first error is reported)	Message box
Name = Asha, Age = abc	Message: "Age must be numeric"; focus to txtAge	Message box
Name = Asha, Age = 120	Message: "Age must be 1 to 100"; focus to txtAge	Message box
Name = Asha, Age = 25	lblResult shows **Welcome Asha, age 25**	Label

Field-level vs form-level

Basis	Field-level	Form-level
Scope	One control at a time	All controls together
When	While typing / when leaving the field	When the user clicks Submit / OK / Save
Events	`KeyPress`, `Change`, `LostFocus`, `Validate`	`Click` event of the command button
Advantage	Immediate feedback; errors caught early	Final check; catches skipped or pasted data and cross-field rules
Limitation	User may never visit some fields; can interrupt typing	Errors found late; several fields may need correction

OAVS EXAM POINTS

- Field-level = per control, immediate; form-level = whole form, on button **Click**.
- Use `Exit Sub` / `Exit Function` after the first failure so that only one message appears at a time.
- A reusable `Boolean` function (`IsFormValid`) keeps `Click` code short.
- Consistency checks (e.g. password = confirm password) are naturally done at **form level**.

COMMON MISTAKES / CONFUSING POINTS

- Assuming field-level checks make form-level checks unnecessary; pasted text and skipped fields still need the final check.
- Forgetting to stop processing after an error, so invalid data is processed anyway.

SUMMARY. Field-level validation checks each control as the user works; form-level validation checks everything when Submit is clicked. Use both.

5. Text box Properties

Definition and core concept. The TextBox is the main input control. Only the properties useful for validating and processing input are given below.

Property	Purpose	Default	Use in validation / processing
`Text`	Holds the data typed; always a String; **default property**	Control name (e.g. Text1)	Read it to validate; set to "" to clear
`MaxLength`	Maximum characters the user can enter	0 (no limit set)	Length check, e.g. 4-digit PIN
`Locked`	True: user cannot edit, but can select/copy; code can still change it	False	Read-only fields such as results
`Enabled`	False: dimmed, cannot get focus or input	True	Block a field until a condition is met
`PasswordChar`	Character displayed instead of each typed character; `Text` keeps the real text	"" (none)	Password entry, e.g. "*" (ignored if `MultiLine` = True)
`MultiLine`	Allows several lines of text	False	Comments / address fields (with `ScrollBars`)
`CausesValidation`	Whether moving focus **to** this control fires Validate of the control being left	True	Set False for Cancel / Help buttons (Topic 7)
`SelStart`, `SelLength`, `SelText`	Start position, length and text of the selection (run-time only)	0	Highlight wrong entry for re-typing
`TabIndex`, `TabStop`	Order of focus with Tab key; whether Tab can reach the control	Creation order; True	Logical data-entry order
`ToolTipText`	Hint shown when the mouse rests on the control	"" (none)	Tell the user the expected format

Method: `SetFocus` moves the cursor to the control (used to send the user back to an invalid field).

Programs with output

PROGRAM 9: Setting validation-related properties at run time

Controls on the form: txtPin, txtPass, txtResult (TextBoxes)

```
Private Sub Form_Load()
    txtPin.MaxLength = 4
    txtPass.PasswordChar = "*"
    txtResult.Locked = True
End Sub
```

OUTPUT

Input / action	Output displayed	Shown in
Types 12345 in txtPin	txtPin shows **1234**; the fifth character is rejected	Text box
Types abcd in txtPass	Box displays ********; txtPass.Text still returns "abcd"	Text box
Types in txtResult	No change: the box is read-only (text can still be selected and copied)	Text box
Code runs txtResult.Text = "OK"	txtResult shows **OK** (Locked stops the user, not the code)	Text box

PROGRAM 10: Highlighting a wrong entry (SelStart and SelLength)

Controls on the form: txtAge (TextBox), cmdHighlight (CommandButton)

```
Private Sub cmdHighlight_Click()
    txtAge.SetFocus
    txtAge.SelStart = 0
    txtAge.SelLength = Len(txtAge.Text)
End Sub
```

OUTPUT

Input / action	Output displayed	Shown in
txtAge = abc, click Highlight	Focus moves to txtAge and the whole text **abc** is highlighted	Text box
User then types 30	Highlighted text is replaced: txtAge shows **30**	Text box

PROGRAM 11: Text is a String: joining vs adding

Controls on the form: Text1, Text2 (TextBoxes), cmdCalc (CommandButton), lblJoin, lblSum (Labels)

```
Private Sub cmdCalc_Click()
    Text1.Text = "10"
    Text2.Text = "20"
    lblJoin.Caption = Text1.Text + Text2.Text
    lblSum.Caption = Val(Text1.Text) + Val(Text2.Text)
End Sub
```

OUTPUT

Input / action	Output displayed	Shown in
Click Calc	lblJoin shows **1020** (the two strings are joined)	Label
Click Calc	lblSum shows **30** (Val converts, then the numbers are added)	Label

OAVS EXAM POINTS

- **Text** is the default property: `Text1 = "Hi"` means `Text1.Text = "Hi"`.
- **Locked** = read-only but selectable; **Enabled = False** = dimmed and no focus.
- `SelStart`, `SelLength` and `SelText` are available only at **run time**.
- `MaxLength = 0` means no limit has been set by the property.

COMMON MISTAKES / CONFUSING POINTS

- **PasswordChar** only masks the display; it does not encrypt or change `Text`.
- Confusing **Locked** and **Enabled**.
- `+` on two TextBox values joins strings; convert with `Val` before adding.
- Believing `MaxLength` is enough validation; it sets only an upper limit on typed length.

SUMMARY. Key validation properties: Text, MaxLength, Locked, Enabled, PasswordChar, MultiLine, CausesValidation, SelStart/SelLength/SelText, TabIndex and ToolTipText.

6. Using Events

Definition and core concept. An **event** is an action recognised by a control (a key press, a change of focus, a click). Code written in an **event procedure** (`Private Sub ControlName_EventName()`) runs automatically when the event occurs. Event-driven validation means the checks are placed in the event that fires at the right moment.

Purpose. To validate and process input at exactly the right time: on every key, on change of text, when focus moves or when the form is submitted.

Event	Occurs when	Argument	Use in validation / processing
`KeyDown` / `KeyUp`	A key is pressed / released	`KeyCode`, `Shift`	Detect special keys (arrows, function keys, Delete)
`KeyPress`	An ASCII character key is pressed	`KeyAscii`	Filter characters; `KeyAscii = 0` cancels the key
`Change`	Contents of the text box change (typing, paste or code)	None	Live processing: character count, enable a button
`GotFocus`	Control receives focus	None	Select existing text, show a hint
`LostFocus`	Control loses focus	None	Field-level check after leaving
`Validate`	Focus is about to leave for a control with `CausesValidation` = True	`Cancel`	Preferred field-level check (Topic 7)
`Click` (CommandButton)	Button is clicked	None	Form-level validation, then processing
`Form_Load`	Form is loaded	None	Initial settings such as `MaxLength`

Event sequence

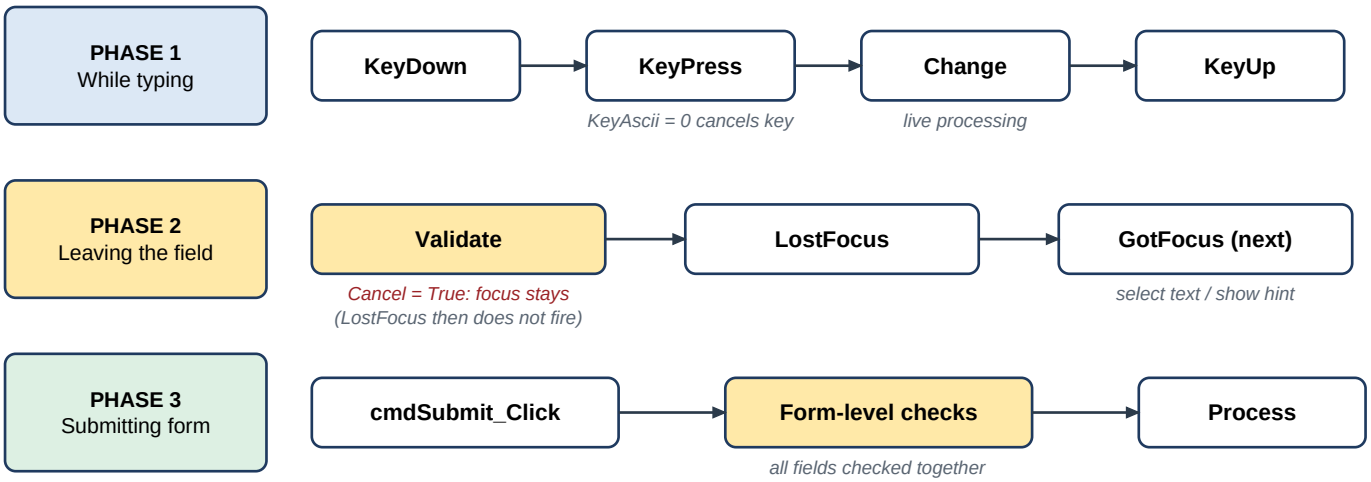

Figure 4: Event-driven validation flow

Order: typing = `KeyDown`, `KeyPress`, `Change`, `KeyUp`; leaving a field = `Validate`, `LostFocus`, then `GotFocus` of the next control.

Common key codes: 8 Backspace (`vbKeyBack`), 13 Enter (`vbKeyReturn`), 32 Space, 48-57 digits 0-9, 65-90 A-Z, 97-122 a-z.

Programs with output

PROGRAM 12: Letters and space only (KeyPress event)

Controls on the form: txtName (TextBox)

```
Private Sub txtName_KeyPress(KeyAscii As Integer)
    If Not ((KeyAscii >= 65 And KeyAscii <= 90) Or _
            (KeyAscii >= 97 And KeyAscii <= 122) Or _
            KeyAscii = 32 Or KeyAscii = 8) Then
        KeyAscii = 0
    End If
End Sub
```

OUTPUT

Input / action	Output displayed	Shown in
Types A, 1, b, 2	txtName shows **Ab** (digits ignored)	Text box
Types R@vi Kumar	txtName shows **Rvi Kumar** (@ ignored, space allowed)	Text box
Presses Backspace	Last character is deleted (code 8 is allowed)	Text box
Types only digits	No change: txtName stays empty	Text box

PROGRAM 13: Live character counter (Change event)

Controls on the form: txtComment (TextBox, MaxLength = 100), lblCount (Label)

```
Private Sub txtComment_Change()
    lblCount.Caption = Len(txtComment.Text) & " / 100"
End Sub
```

OUTPUT

Input / action	Output displayed	Shown in
Types Hello	lblCount shows **5 / 100**	Label
Deletes 2 characters	lblCount shows **3 / 100**	Label
Clears the box	lblCount shows **0 / 100**	Label
Types a 101st character	Rejected by MaxLength; lblCount stays **100 / 100**	Label

PROGRAM 14: Select existing text on entry (GotFocus event)

Controls on the form: txtName, txtAge (TextBoxes)

```
Private Sub txtAge_GotFocus()
    txtAge.SelStart = 0
    txtAge.SelLength = Len(txtAge.Text)
End Sub
```

OUTPUT

Input / action	Output displayed	Shown in
txtAge holds 25; user presses Tab into it	Text **25** is highlighted	Text box
User types 3	Highlighted text is replaced: txtAge shows **3**	Text box

OAVS EXAM POINTS

- **KeyPress** gives `KeyAscii`; **KeyDown/KeyUp** give `KeyCode`.
- Setting `KeyAscii = 0` **discards** the keystroke.
- `Form.KeyPreview = True` lets the form's key events see keystrokes before the control.
- Validate fires **before** LostFocus.

COMMON MISTAKES / CONFUSING POINTS

- **KeyPress cannot detect** arrow or function keys, and it does **not** catch pasted text.
- Forgetting to allow **Backspace** (8) in a KeyPress filter.
- Changing `Text` inside its own `Change` event triggers the event repeatedly.
- `Change` also fires when code assigns `Text`, not only when the user types.

SUMMARY. Use KeyPress to filter keys, Change for live processing, GotFocus for preparation, LostFocus/Validate for field checks and Click for form-level checks.

7. Validate Event

Definition and core concept. The **Validate event** (introduced in VB 6.0) occurs **before a control loses focus**, letting the program check the data and decide whether focus may leave. It is available for controls that have the `CausesValidation` property.

Purpose. To hold the user in an invalid field until correct data is entered, without the focus problems of `LostFocus` + `SetFocus`.

Syntax: `Private Sub object_Validate(Cancel As Boolean)`

- **Cancel = False** (default): focus moves to the next control.
- **Cancel = True**: focus **stays** in the current control (no `SetFocus` needed).

Working

- The user presses Tab or clicks another control.
- VB checks the **CausesValidation** property of the control that is **receiving** focus.
- If True, the `Validate` event of the control being left runs; your code tests the data.
- If the code sets `Cancel = True`, focus stays; otherwise `LostFocus` fires and focus moves on.

Program with output

PROGRAM 15: Validate event: age check with a Cancel button

Controls on the form: txtAge, txtCity (TextBoxes), cmdCancel (CommandButton, CausesValidation = False)

```
Private Sub txtAge_Validate(Cancel As Boolean)
    If Not IsNumeric(txtAge.Text) Then
        MsgBox "Age must be a number", vbExclamation
        Cancel = True
    ElseIf Val(txtAge.Text) < 1 Or Val(txtAge.Text) > 100 Then
        MsgBox "Age must be between 1 and 100", vbExclamation
        Cancel = True
    End If
    If Cancel Then
        txtAge.SelStart = 0
        txtAge.SelLength = Len(txtAge.Text)
    End If
End Sub

Private Sub cmdCancel_Click()          ' cmdCancel.CausesValidation = False at design time
    Unload Me
End Sub
```

OUTPUT

Input / action	Output displayed	Shown in
Types abc in Age, presses Tab	Message: "Age must be a number"; focus stays in txtAge with text highlighted	Message box
Types 150, presses Tab	Message: "Age must be between 1 and 100"; focus stays in txtAge	Message box
Types 25, presses Tab	No message; focus moves to txtCity	No output
Leaves abc in Age, clicks Cancel	Validate does **not** fire; the form closes	Form closes

Validate vs LostFocus

Basis	Validate	LostFocus
Occurs	**Before** focus leaves	**After** focus has left
Can stop focus leaving?	Yes, `Cancel = True`	No (needs `SetFocus` workaround)
Fires when target has `CausesValidation` = False?	No	Yes
Argument	`Cancel As Boolean`	None
Introduced	VB 6.0	Earlier versions also

OAVS EXAM POINTS

- Syntax: `object_Validate(Cancel As Boolean)`; **Cancel = True** keeps focus.
- The **CausesValidation of the control receiving focus** decides whether Validate fires.
- Set `CausesValidation = False` on Cancel / Help / Exit buttons so users can always escape.
- The form method `ValidateControls` fires the Validate event of the control that currently has focus.
- Event order: `Validate` -> `LostFocus` -> `GotFocus` (next control).

COMMON MISTAKES / CONFUSING POINTS

- Setting `CausesValidation = False` on the **invalid text box** instead of on the Cancel button.
- Showing the message but forgetting `Cancel = True`: the focus still moves away.
- Leaving Cancel button with `CausesValidation = True`: the user is trapped in the invalid field.
- Thinking Validate is the same as LostFocus; Validate runs **earlier** and can **cancel** the move.

SUMMARY. Validate runs before focus leaves a control; setting Cancel = True keeps focus there. It fires only when the next control has CausesValidation = True, so set it to False on Cancel buttons.

QUICK REVISION

Key definitions

Term	Meaning
Validation	Checking input for completeness, type, range and format before use
Processing	Converting, calculating, storing or displaying the accepted data
Field-level validation	Checks one control while typing or on leaving it
Form-level validation	Checks all controls together when Submit / OK is clicked
Validate event	Fires before focus leaves a control; `Cancel = True` keeps focus there
GIGO	Garbage In, Garbage Out

Validation types and tools

Type	Tool	Type	Tool
Presence	`Len(Trim())`	Format	`Like`, `InStr`
Data type	`IsNumeric`, `IsDate`	Character	`KeyPress`, `KeyAscii`
Range	Comparison with `Val`	List	`Select Case`
Length	`Len`, `MaxLength`	Consistency	Compare related fields

Important properties and events

Properties	Events
`Text` (default), `MaxLength`, `Locked`, `Enabled`, `PasswordChar`, `MultiLine`, `CausesValidation`, `SelStart`/`SelLength`/`SelText`, `TabIndex`, `ToolTipText`. Method: `SetFocus`	`KeyDown`, `KeyPress` (`KeyAscii`), `KeyUp`, `Change`, `GotFocus`, `Validate` (`Cancel`), `LostFocus`, `Click`. Typing: KeyDown, KeyPress, Change, KeyUp. Leaving: Validate, LostFocus, GotFocus.

Likely exam points

- Event that can stop focus from leaving a control: **Validate** (set **Cancel = True**).
- Property deciding whether Validate fires: **CausesValidation** of the control receiving focus.
- Cancel button should have **CausesValidation = False**.
- **KeyAscii = 0** cancels a keystroke; KeyPress cannot detect arrow/function keys or pasted text.
- Mask typed characters: **PasswordChar**; limit characters: **MaxLength**; read-only: **Locked = True**.
- Default property of TextBox: **Text** (always a String).
- Form-level validation goes in the command button's **Click** event.
- Errors: **13** Type mismatch; **11** Division by zero; **6** Overflow.
- `Val("abc")` = 0 and `Val("12abc")` = 12; Val converts, it does not validate.
- `IsNumeric` returns **Boolean**; it accepts "1e5" and "$25".
- Validation checks reasonableness, **not** truth; verification is different (e.g. re-entry).